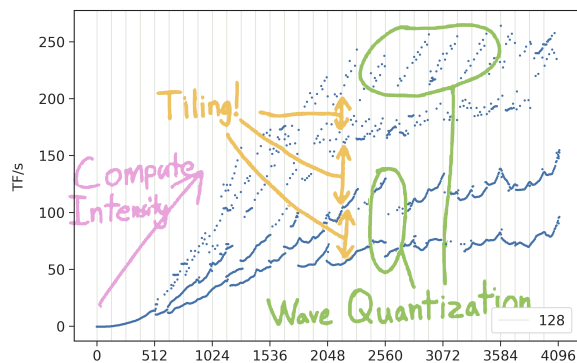


Lecture 5: GPUs (and TPUs)

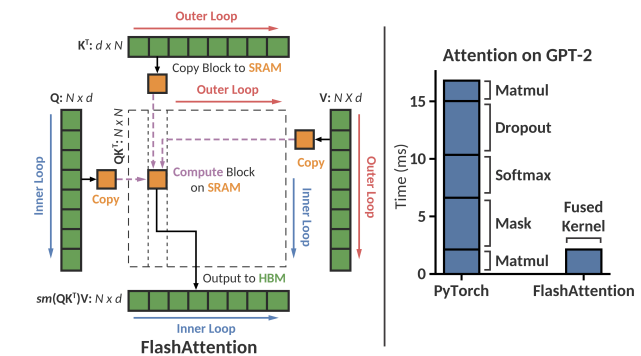
Engineering the Graphics Processing Unit

Goal: Make GPUs (and TPUs) less magic.

Understand when GPUs get slow.

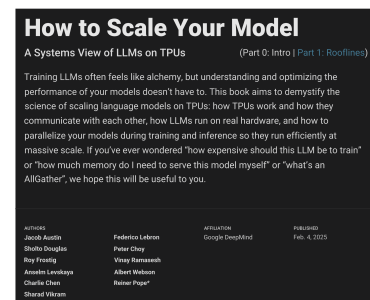
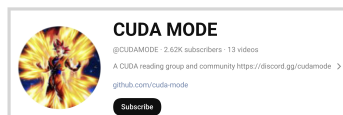
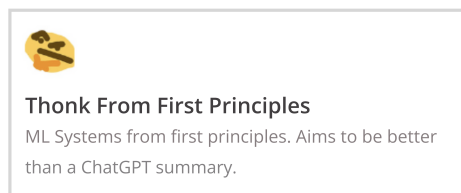


Understand how to make fast algorithms.



Before we start..

Substantial credit goes to a few sources I'd like to highlight ..



Horace He's blog (!)

GPU Mode group (!)

TPU (and now GPU) book (!!)

Organization today:

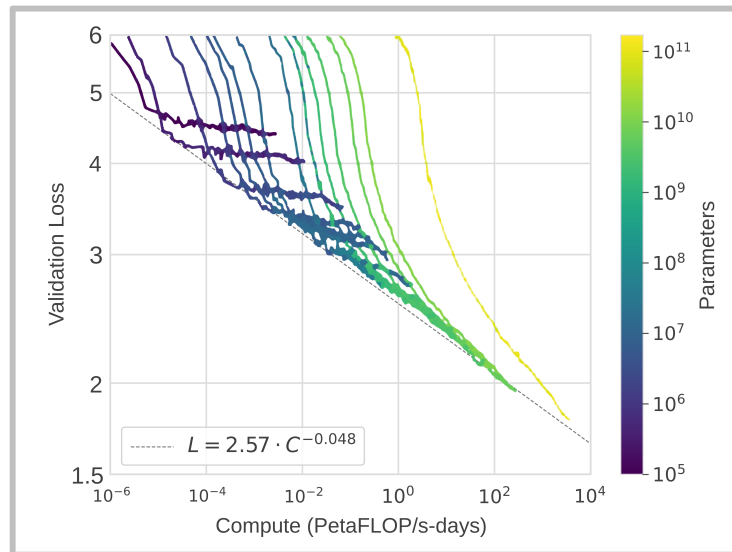
Part 1: GPUs in depth - how they work and important parts.

Part 2: Understanding GPU performance.

Part 3: Putting it together - unpacking FlashAttention.

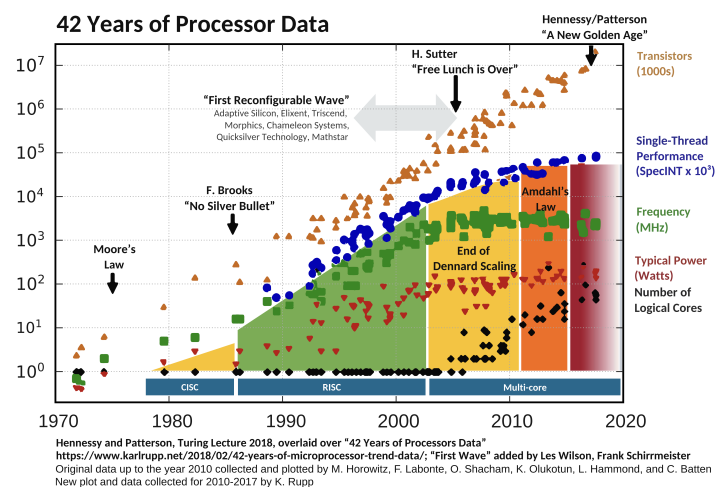
Setting the stage: compute leads to predictable performance

Often times, compute leads to predictable accuracy gains for language models.



Faster hardware, better utilization, improved parallelization can drive progress (for now).

How do we get compute scaling? Early on - Dennard scaling



But the traditional form of scaling (*Dennard scaling*) from 1980-2000s has tapped out ... how do we feed LLMs' insatiable appetite for compute?

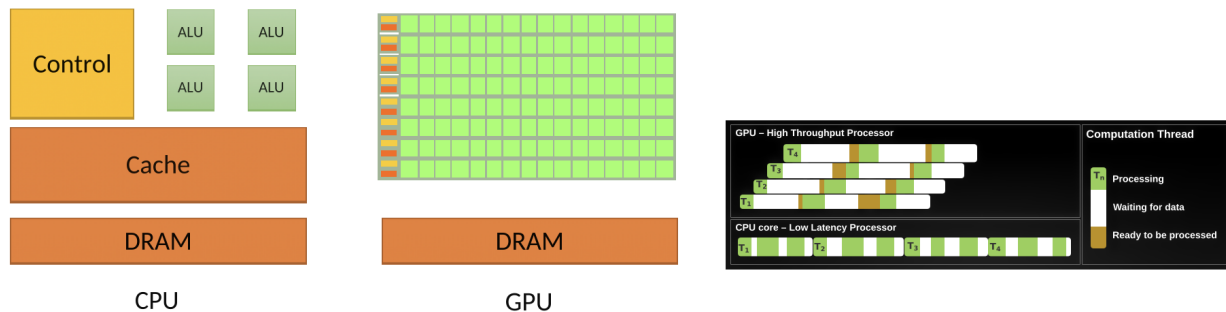
Parallel scaling continues

Parallel scaling with GPUs has scaled $> 1000\times$ in 10 years.

Claim: *There is no LLM scaling without GPU scaling.*

How is a CPU different from a GPU?

CPUs optimize for few fast threads WHILE GPUs optimize for many many threads.



Many tiny ALUs.
Much less support for branching.

GPUs optimize for throughput.
CPUs optimize for latency.

Anatomy of a GPU (execution units)



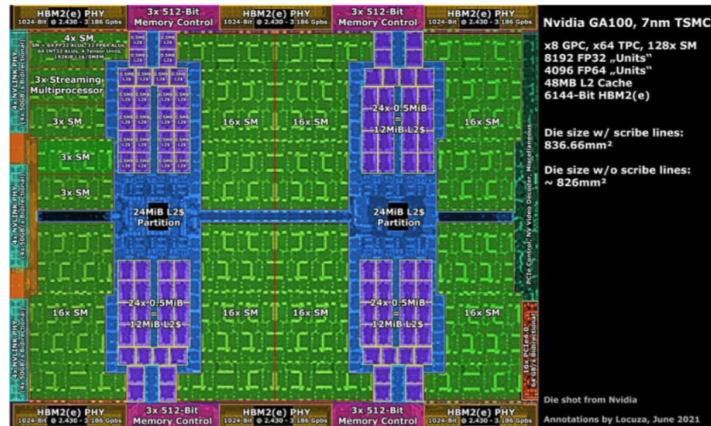
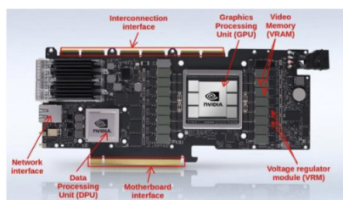
- GPUs have many **SMs (streaming multiprocessors)** that independently execute "blocks" (jobs).
- Each SM further contains many **SPs (streaming processors)** that can execute "threads" in parallel.

Anatomy of a GPU (memory)

The closer the memory to the SM, the faster it is - L1 cache and shared memory is *inside* the SM. L2 cache is on the die and global memory are memory chips next to the GPU.

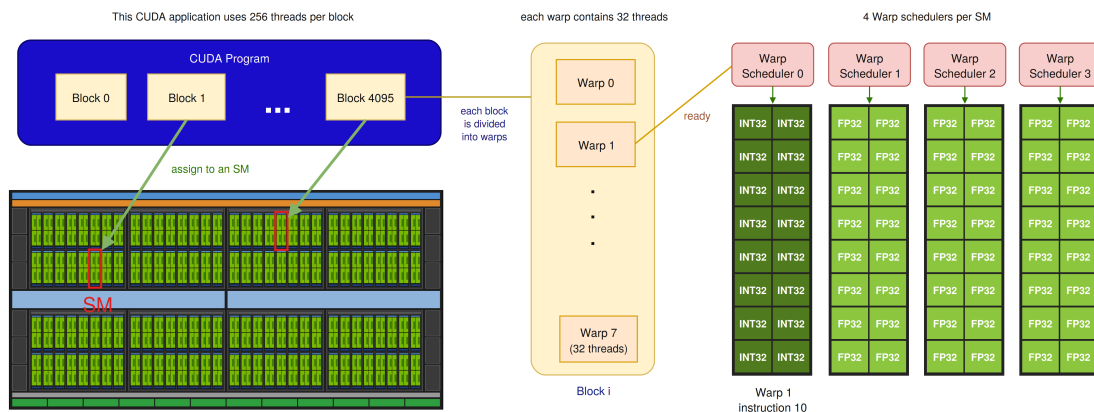
TABLE IV
THE MEMORY ACCESS LATENCIES

Memory type	CPI (cycles)
Global memory	290
L2 cache	200
L1 cache	33
Shared Memory (ld/st)	(23/19)



SRAM (shared/cache memory) is much more expensive (100×) but about 8× faster than DRAM (global memory).

Execution model of a GPU

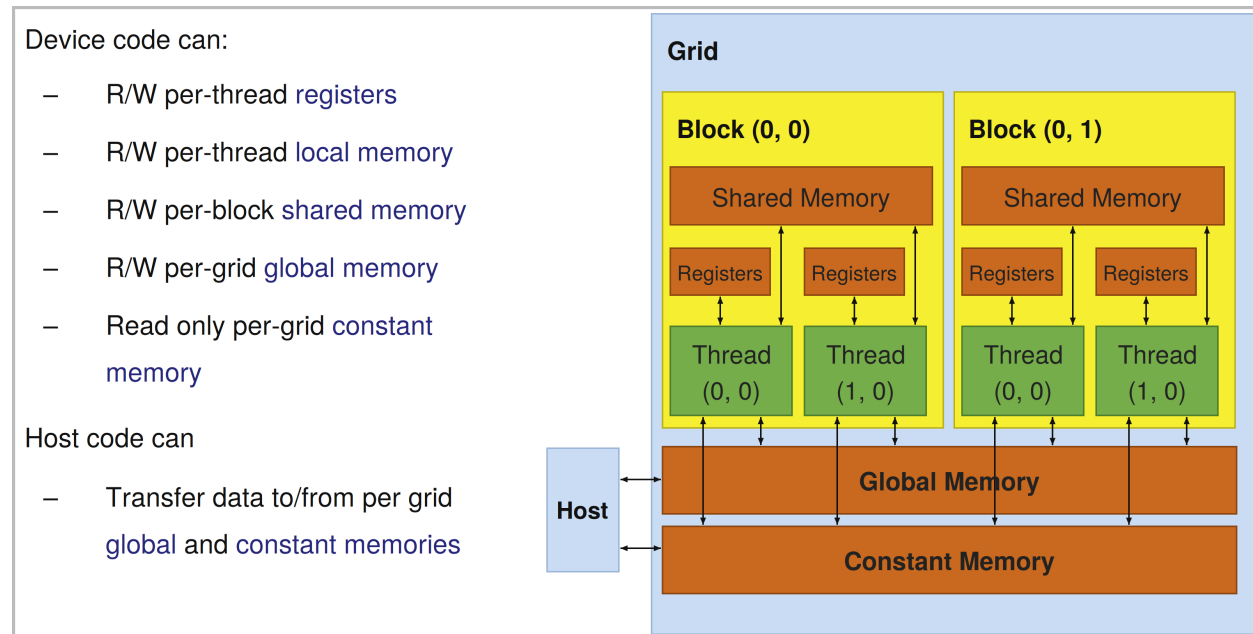


There are 3 important players in this execution model.

- **Threads:** Threads "do the work" in parallel - all threads execute the same instructions but with different inputs (SIMT).
- **Blocks:** Blocks are groups of threads. Each block runs on a SM with its own shared memory.
- **Warp:** Threads always execute in a "warp" of 32 consecutively numbered threads each.

Memory model of a GPU

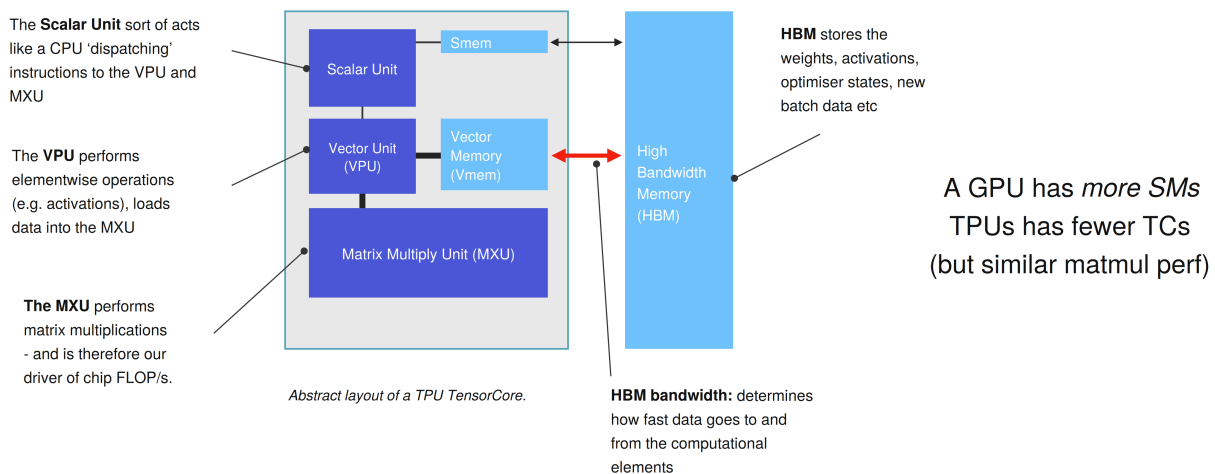
Each thread can access its own registers and shared memory within the block.



Information that goes across blocks needs to be read/written to global memory (slow).

Side thread - What about TPUs?

GPUs, TPUs, and many other accelerators are at a high level, similar.



- **Core structure** - lightweight control, fast (big) matmul unit, fast memory.

GPU	TPU	What is it?
Streaming Multiprocessor (SM)	Tensor Core	Core “cell” that contains other units
Warp Scheduler	VPU	SIMD vector arithmetic unit
CUDA Core	VPU ALU	SIMD ALU
SMEM (L1 Cache)	VMEM	Fast on-chip cache memory
Tensor Core	MXU	Matrix multiplication unit
HBM (aka GMEM)	HBM	High bandwidth high capacity memory

- Execution difference - no warps (just blocks - tradeoffs in matMul vs nonMatMul).
- Networking difference - how accelerators are networked (in parallelism lecture).

Strengths of the GPU model

- Easily scales up hard workloads (by adding more SMs).
- Easy (?) to program due to the SIMT model.
- Threads are “lightweight” and can be stopped/resumed.

GPUs as fast matrix multipliers

- Early days of NVIDIA GPUs - programmable shaders.
- Researchers hacked this to do matmuls.
- Volta days of NVIDIA GPUs - new matmul hardware (means matmuls are fast and special).
- Tensor cores (introduced in V and T series) are specialized matrix multiplication circuits.

Matmuls are $> 10\times$ faster than other floating point ops!

GPUs - what are they and how do they work

GPUs are massively parallel - same instructions applied across many workers.

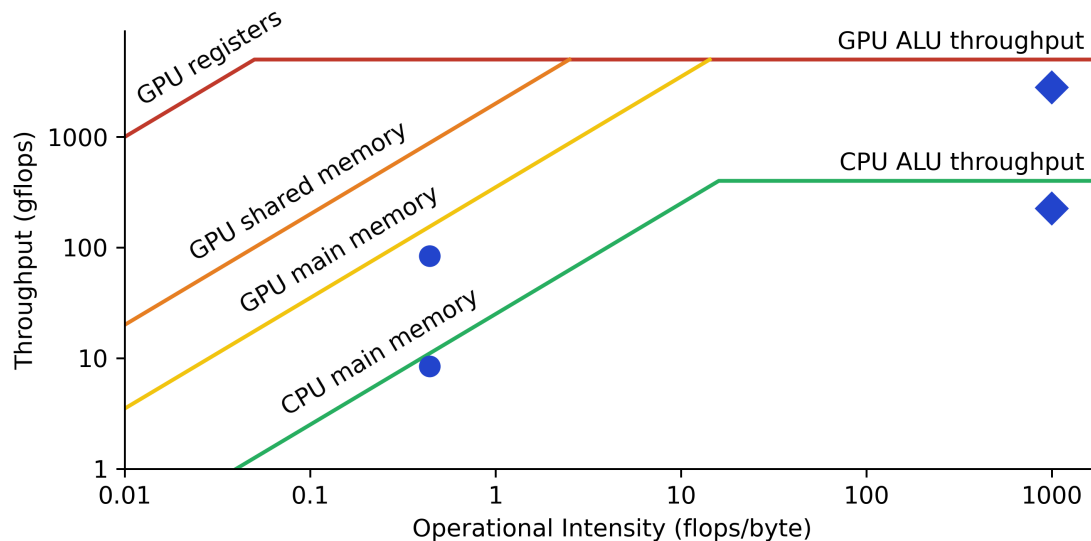
Compute (and especially matmuls) have scaled faster than memory.

We have to respect the memory hierarchy to make things go fast.

Making ML workloads fast on a GPU

Performance on a GPU can be complex, even for something as simple as a square matmul.

- Dense matrix multiply ◆
- Sparse matrix multiply ●



Key to this section: **how do we avoid being memory bound?**

1. Control divergence (not a memory bottleneck)
2. Low precision computation
3. Operator fusion
4. Recomputation
5. Coalescing memory
6. Tiling

Control divergence (not a memory issue)

- GPUs operate in a SIMT model - every thread in a warp is executing the same instruction.
- Conditionals are fine, but lead to significant overhead from the execution model.

Trick 1: Low precision computation

If you have fewer bits, you have fewer bits to move.

Example: elementwise ReLU ($x = \max(0, x)$) on a vector of size n .

(Float 32 case)

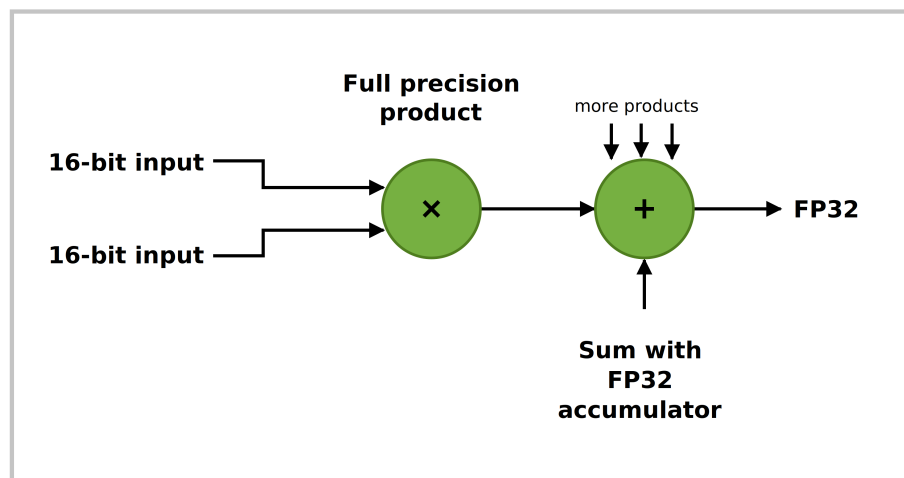
- **Memory access:** 1 read (x), 1 write (if $x < 0$), fp32 = 4 bytes per op.
- **Operations:** 1 comparison op, 1 FLOP.
- **Intensity Inverse:** 8 bytes / FLOPs.

(Float 16 case)

- **Memory access:** 1 read (x), 1 write (if $x < 0$), fp16 = 2 bytes per op.
- **Operations:** 1 comparison op, 1 FLOP.
- **Intensity Inverse:** 4 bytes / FLOPs.

Low precision drives faster matrix multiples

Lots of operations in modern GPUs are accelerated via low / mixed precision operations.

Tensor cores

Operations that can use 16-bit storage (fp16/bf16)

- Matrix multiplication.
- Most pointwise operations (e.g. relu, tanh, add, sub, mul).

Operations that need ore precision (fp32/fp16)

- Adding small values to large sums can lead to rounding errors.
- Reduction operations (e.g. sum, softmax, normalization).

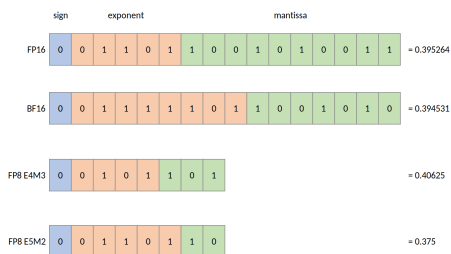
Operations that need more range (fp32/bf16)

- Pointwise operations where $|f(x)| \gg |x|$ (e.g. exp, log, pow).
- Loss functions.

Frontier in low precision

Very low precision (FP8) with different tradeoffs. Multiple scaling factors MXFP8 (Blackwell).

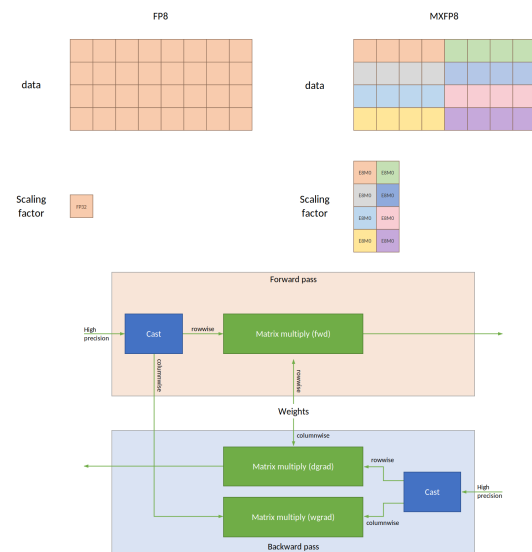
Very low precision (FP8) with different tradeoffs



MXFP8 has many *interesting* things about it

- Uses E4M3 (more mantissa) due to more scale factors
- Scale factors themselves are FP8 (E8M0), 1 per 32
- Transposes are now nontrivial!

Multiple scaling factors MXFP8 (Blackwell)

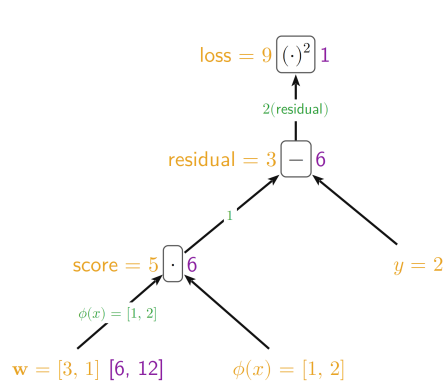


Trick 2: Operator fusion

Think of a GPU like a factory - inputs come from a warehouse (memory) and is processed at a factory.

- What if we have to do many operations? Shipping back and forth is somewhat silly.
- **Example** - Computing $\sin^2 x + \cos^2 x$ naively launches 5 CUDA kernels (back and forth).

All 5 pointwise operations can be fused into a single CUDA kernel call. "Easy" fusions like this can be done automatically by compilers (torch.compile).

Trick 3: Recomputation

$$\text{Loss}(x, y, \mathbf{w}) = (\mathbf{w} \cdot \phi(x) - y)^2$$

$$\mathbf{w} = [3, 1], \phi(x) = [1, 2], y = 2$$

↓ **backpropagation**

$$\nabla_{\mathbf{w}} \text{Loss}(x, y, \mathbf{w}) = [6, 12]$$

**Definition: Forward/backward values**

Forward: f_i is value for subexpression rooted at i

Backward: $g_i = \frac{\partial \text{loss}}{\partial f_i}$ is how f_i influences loss

**Algorithm: backpropagation algorithm**

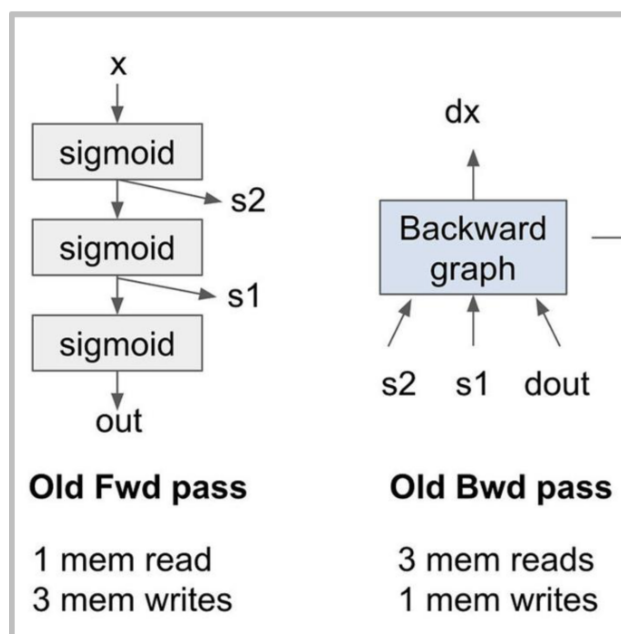
Forward pass: compute each f_i (from leaves to root)

Backward pass: compute each g_i (from root to leaves)

In backpropagation, we store the activations (yellow) and compute Jacobians (green).

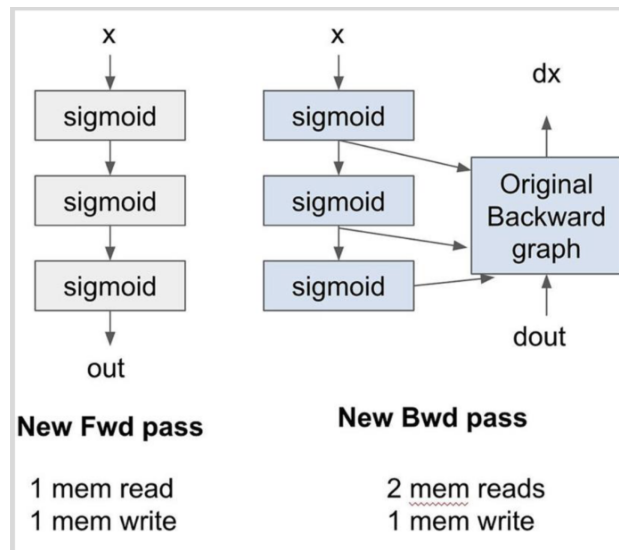
Storing (and retrieving) activations can be expensive!

Let's say we stack 3 sigmoids on top of each other.



This is really terrible for performance - 8 memory read/writes, very low arithmetic intensity.

Throw away the activations, re-compute them!



Throwing away computation can be actually be optimal, with 5/8th the memory accesses!

Trick (?) 4: DRAM and Memory coalescing

DRAM (global memory) is read in "burst mode" - each read gives you many bytes!

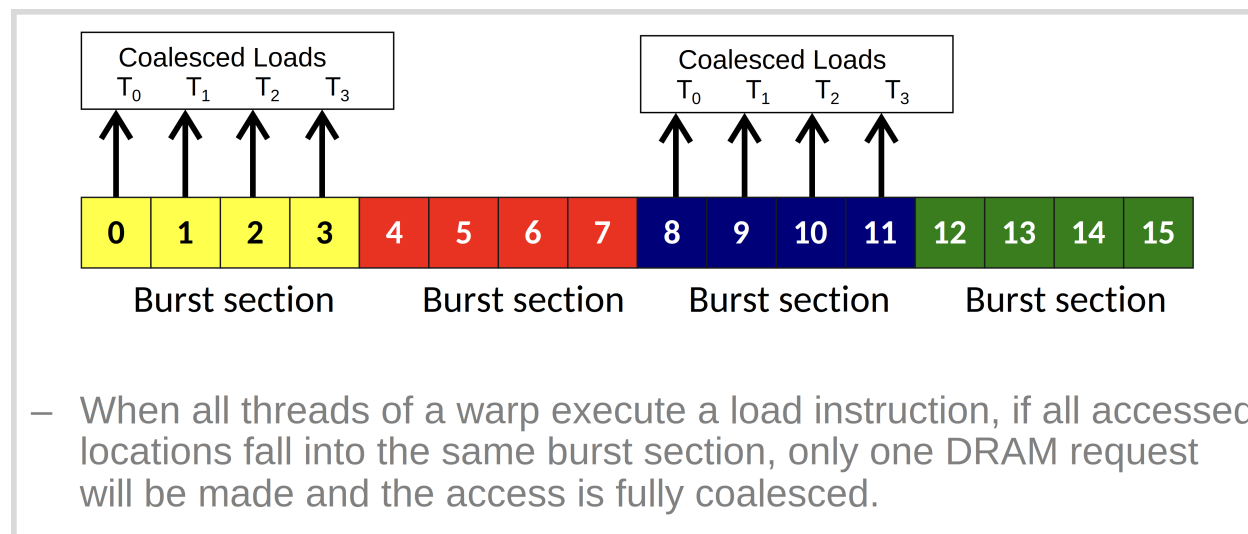
Burst section				Burst section				Burst section				Burst section			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

- Each address space is partitioned into burst sections
 - Whenever a location is accessed, all other locations in the same section are also delivered to the processor
- Basic example: a 16-byte address space, 4-byte burst sections
 - In practice, we have at least 4GB address space, burst section sizes of 128-bytes or more

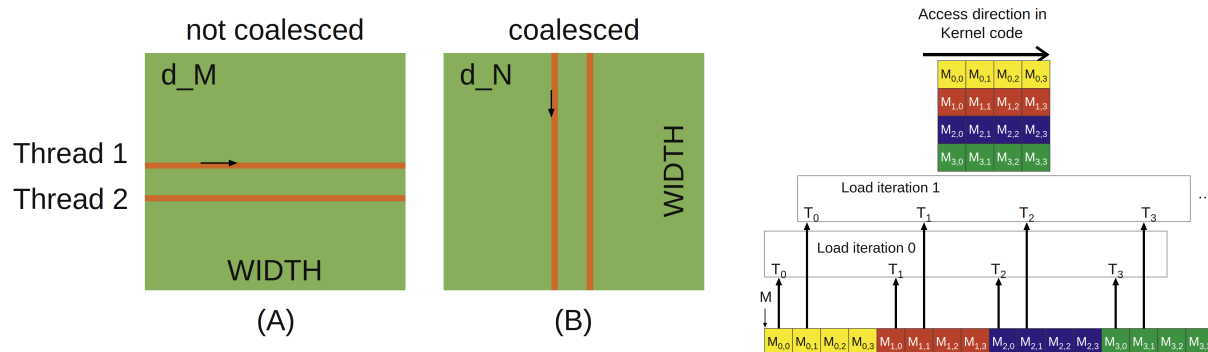
Burst mode comes from the slow per-row copy to the sense amplifier.

Memory coalescing

Memory accesses are *coalesced* if all the threads (in a warp) fall within the same burst.



Coalescing for matrix multiplication



For row-major matrices - **threads that move along rows are not coalesced.**

Remark: Note how the second diagram reads the entire vector at each step!

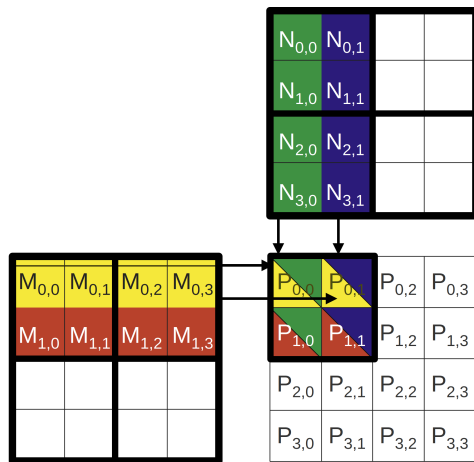
Trick 5 (the big one): Tiling

Tiling is the idea of grouping and ordering threads to minimize global memory access.

Let's go back to matrix multiplication .. Note that memory access is not coalesced and repeated

Tiling - store and reuse information in shared memory

Cut up the matrix into smaller "tiles" and load this into shared memory.



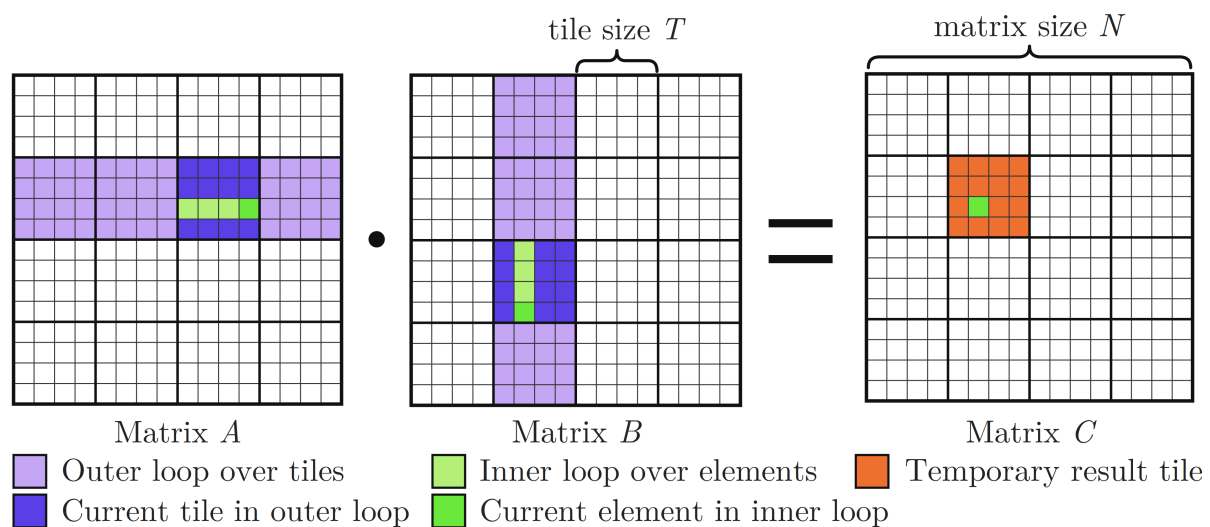
Compute the matrix multiply in 'phases'

1. Load $M_{0,0}$ and $N_{0,0}$ tiles into SHM
2. Compute partial sums for P
(Done with one tile)
3. Load the $M_{0,0}$ and $N_{2,0}$ tile into SHM
4. ...

Advantages: repeated reads now access shared, not global memory and memory access can be coalesced.

Tiling math

Non-tiled matrix multiply: each input is read N times from global memory.

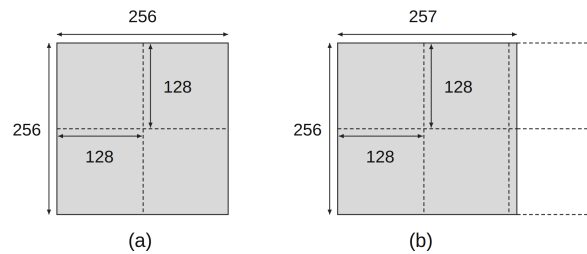


Tiled matrix multiply: each input is read N/T times from global memory and T times within each tile. This is a factor of T reduction in global memory access

Complexities with tiling

Tile sizes may not divide the matrix dimensions and lead to low utilization.

Figure 6. Example of tiling with 128x128 thread block tiles. (a) Best case - matrix dimensions are divisible by tile dimensions (b) Worse case - tile quantization results in six thread blocks being launched, two of which waste most of their work.

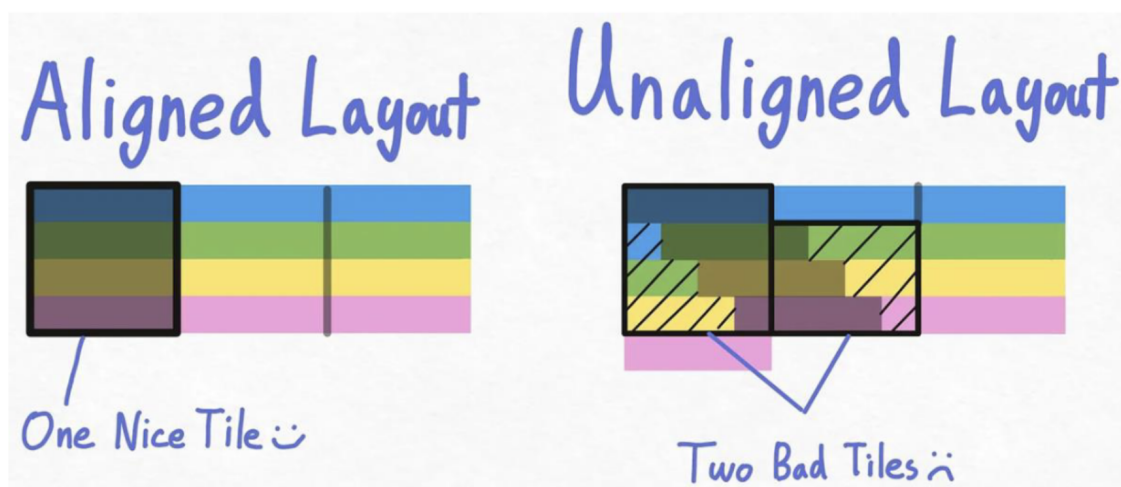


Factors affecting tile sizes:

- Coalesced memory access.
- Shared memory size.
- Divisibility of the matrix dimension.

Complexities with tiling 2 - memory alignment

Loading tiles are fast if bursts align with the matrix.



Coalesced accesses may be impossible depending on the dimension of the matrix .. (have to do padding).

Putting it together: understanding a matrix mystery

Why is it *faster* to have bigger matrices?

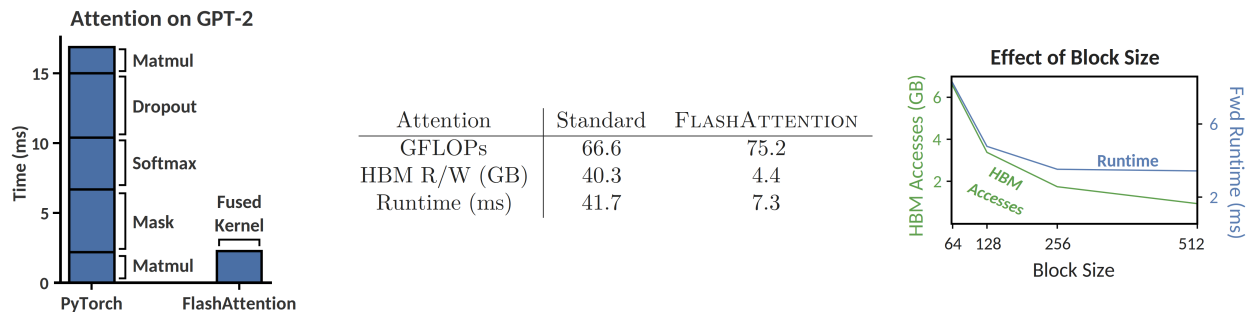
We understand some of this (compute intensity, tiling blocks). Let's take a closer look ..

- Tiling has a major impact through alignment.
- Quantization has a nontrivial impact on periodic behavior.

An A100 has 108 SMs, so it cannot execute all 120.

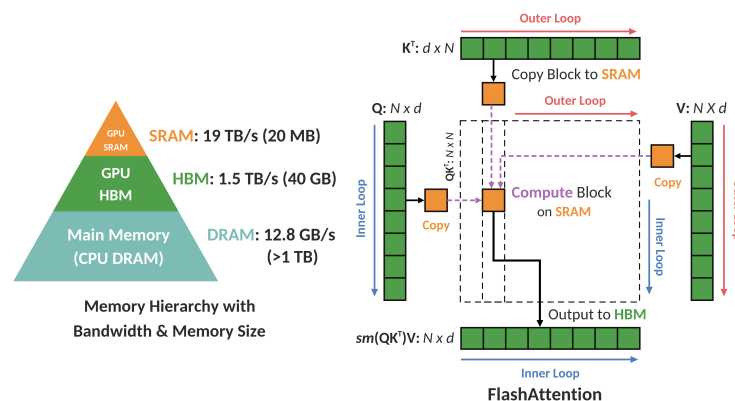
Understanding Flash Attention

Flash attention [Dao et al] dramatically accelerates attention .. but how?



Technique from paper: We apply two established techniques (tiling, recomputation) to overcome the technical challenge of computing exact attention in sub-quadratic HBM accesses.

Part 1: tiling for the KQV matrix multiply



This figure from the paper is literally just tiling for a KQV matrix multiply .. **but what do we do about the softmax?**

Part 2: incremental computation of the softmax

From Mikailov and Gimelshein 2018,

Algorithm 3 Safe softmax with online normalizer calculation

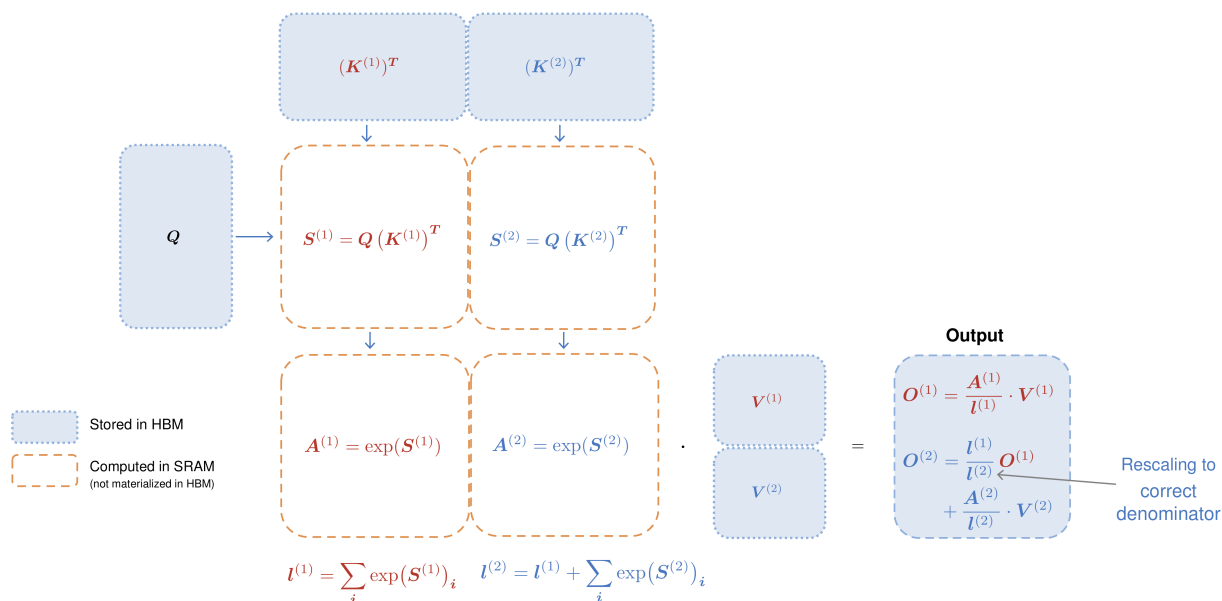
```

1:  $m_0 \leftarrow -\infty$ 
2:  $d_0 \leftarrow 0$ 
3: for  $j \leftarrow 1, V$  do
4:    $m_j \leftarrow \max(m_{j-1}, x_j)$ 
5:    $d_j \leftarrow d_{j-1} \times e^{m_{j-1}-m_j} + e^{x_j-m_j}$ 
6: end for
7: for  $i \leftarrow 1, V$  do
8:    $y_i \leftarrow \frac{e^{x_i-m_V}}{d_V}$ 
9: end for
  
```

To keep track of the max, incrementally update the max and set up a telescoping sum.

This lets you compute the softmax tile-by-tile.

Putting it all together - the forward pass of flash attention



From Dao 2023, we see

- Tile-wise computation of the inner products (S).

- Fusion of the exponential operator.
- Tile-wise computation of the softmax via the online, telescoping sum trick.

(We won't cover the backward pass - but they recompute tile-by-tile ..)

Recapping the Graphics Processing Unit

Hardware powers scale and low-level details determine what scales or doesn't.

Current GPU-based compute strongly encourages thinking about matmul + data movement.

Thinking about the GPU (coalescing, tiling, fusion) leads us to good performance.